
Resident multi-request shared-document reuse for Q16 vLLM: a reproducible audit on Qwen3.5-35B

Anonymous authors
Paper under double-blind review

Abstract

We report a controlled, reproducible study of resident-request sharing for long-context inference with Qwen3.5-35B-A3B under strict same-kernel constraints. Under single-stream, single-batch execution, Q16 precision, and fixed geometry and input governance, we compare two ownership strategies over resident counts $N = \{1, 2, 4, 8, 16, 32\}$: fresh full-document duplication and shared-document reuse.

We use the same PG-19 train-only windows, one fixed document of 4,095 tokens (worst-case partial-tail), the same tokenizer model layout, and synchronized environment/build settings. Our findings are two-fold:

- Across all N , both arms produce exact token and full-vocab-logit trajectories, with exact final logical K/V and GDN state.
- The shared-document design reduces resident parse-book scaling linearly: each additional resident request saves about 85 MiB (80 MiB from avoided document copy, 5 MiB from per-request private state structure), with controlled memory scaling across N up to 32.

This manuscript only claims correctness-preserving memory-structure effects for the closed experiment slice above. We do not claim improvements in throughput, latency, scheduler behavior, ragged batching, multi-document generalization, Q8/Q4 behavior, or downstream quality (F1/EM) changes.

Contributions:

- A reproducible 8-rank multi-resident experimental suite for fixed one-kernel, same-batch inference.
- Raw-level replayed validity checks for token, full-vocab logits, logical K/V, and GDN state.
- A fully auditable parse-book scaling model with explicit provenance and protocol gating.

1 Introduction

When the number of resident requests grows, resident-query systems often duplicate more state than necessary if each request stores a full physical copy of the same long document. Our goal is to test whether shared-document reuse can reduce parse-book growth while preserving exactness in a fixed-scope kernel setup.

Earlier passes had verified same-kernel exactness for single request. The remaining gap was a systematic, reproducible scan of resident count $N > 1$ and corresponding capacity-consistency tradeoffs.

This work studies:

- one process and one model invocation path (single kernel family),
- one stream, batch size 1,
- Q16 precision,

054
055
056
057
058
059
060
061
062
063
064
065
066
067
068
069
070
071
072
073
074
075
076
077
078
079
080
081
082
083
084
085
086
087
088
089
090
091
092
093
094
095
096
097
098
099
100
101
102
103
104
105
106
107

- PG-19 train-only validation.

We report results only for this scope and explicitly avoid global deployment claims.

2 Related Work

This work is positioned as a systems reproducibility study of memory sharing under fixed execution semantics.

- Transformer attention is the base building block (Vaswani et al., 2017).
- FlashAttention reduces attention memory-access overhead through tiled kernels (Dao et al., 2022).
- vLLM’s paged attention exposes practical cache reuse paths but does not solve all protocol or reproducibility requirements by itself (vLLM Team, 2024).

Our contribution is not a new kernel. It is a reproducible evidence chain showing how shared-document reuse scales for resident requests under strict protocol and raw-level replay gates.

3 Methodology

3.1 Experiment design

We compare two arms:

- Fresh baseline. Shared prefill and pack are fixed, then each request gets an independent full document + private region.
- Shared-document reuse. The document block remains shared in a read-only source arena; each request owns only private pages for partial-tail copy-on-write and append state.

3.2 Model and execution

Model and kernel:

- Qwen3.5-35B-A3B, 40 layers, full-attention layers at indices {3, 7, 11, 15, 19, 23, 27, 31, 35, 39} (10 layers).
- vLLM 0.26 unified attention with Triton kernel, batch size $q = 1$, and Q16 execution.
- page size 128, head dimension 256, full attention layers $n_{\text{layer}} = 10$.
- Environment: PyTorch 2.11.0+cu129, Transformers 5.14.1, Triton 3.6.0, FlashInfer 0.6.14.

We run

$$N \in \{1, 2, 4, 8, 16, 32\}, \quad q = 32, \quad g = 8, \quad d = 4095, \quad n_{\text{layer}} = 10.$$

3.3 Data governance

Each rank uses a different PG-19 train window. For each rank we fix 32 non-overlapping 32-token queries per book (stride 64), no synthetic query markers, and forbid source6-9, source68-99, or test-v2 reads in this experiment. All windows, query banks, and train-only objects are hashed and checked against frozen manifest identities.

The document length is fixed at 4,095 tokens ($\text{mod } 128 = 127$), making partial-tail behavior explicit and providing a conservative stress condition.

3.4 Capacity units

For one layer in this setting,

$$C_{32}^{\text{layer}} = 2,621,440 \text{ bytes.}$$

For 10 full-attention layers, total is $10 \times C_{32}^{\text{layer}} = 26,214,400$ bytes (≈ 25 MiB). All capacity numbers used below are for these 10 full-attention layers (not per-layer values).

Let:

$$\text{fresh}(N) = 80 \text{ MiB} + 90N \text{ MiB}, \quad \text{reuse}(N) = 80 \text{ MiB} + 5N \text{ MiB}, \quad \Delta(N) = 85N \text{ MiB}.$$

Here 80 MiB reflects shared source document copy avoidance, and $5N$ MiB is private reservation growth in shared mode.

4 Correctness protocol

For each raw shard, we record trajectories and enforce replay in aggregate:

- full token sequence,
- full-vocab logit tensors,
- final logical K/V,
- GDN state (60 tensors, full exact checks),
- call descriptors and kernel identity.

We reject reports that pass on aggregate booleans alone. The replay pipeline recomputes medians and validates layer, protocol, query provenance, allocator baselines, and storage formulas from raw measurements.

Observed invariants.

- Fresh and reuse are exact for token and full-logit outputs (ratio 1.0).
- Final logical K/V and GDN state are exact between arms.
- No dense fallback or full-document cat is used.
- Cross- N prefix isolation holds: the same request index is stable across resident-counts.

HF-based references are treated as non-blocking diagnostics only and are not used as the primary deployment claim.

5 Capacity and resource results

Closed-form parse-book scaling in 10-layer full-attention scope:

N	fresh (MiB)	reuse (MiB)	controlled savings (MiB)	avoided doc copy (MiB)
1	170	85	85	80
2	260	90	170	160
4	440	100	340	320
8	800	120	680	640
16	1520	160	1360	1280
32	2960	240	2720	2560

Table 1: Median parse-book scale (10 full-attention layers) over 8 ranks.

These are parse-book quantities only and are not equivalent to total GPU memory.

For allocator-based reporting, we use PyTorch allocator allocated/reserved (not NVML peaks). All reported values are medians over 8 ranks, with raw replays as the source of truth.

At $N = 32$, absolute production peak allocated drops from 74,623,183,360 bytes (fresh) to 71,765,915,136 bytes (reuse), a reduction of 2,857,268,224 bytes (≈ 2.661 GiB).

N	fresh setup+generation current delta	reuse setup+generation current delta	difference	absolute alloc
1	147.135 MiB	61.883 MiB	85.252 MiB	64.896
2	298.522 MiB	126.766 MiB	171.757 MiB	65.048
4	591.286 MiB	249.520 MiB	341.767 MiB	65.345
8	1177.831 MiB	495.045 MiB	682.786 MiB	65.936
16	2355.406 MiB	990.081 MiB	1365.325 MiB	67.126
32	4705.065 MiB	1980.162 MiB	2724.903 MiB	69.498

Table 2: Current allocated deltas and peak allocated (8-rank medians).

6 Reproducibility and audit

Execution used Job 237580 / Trial 1840837 and completed successfully. Stage order: 00_start $\rightarrow$ 01_static_preflight_ok $\rightarrow$ 02_resident_shards_ok $\rightarrow$ 99_done, with no FAILED artifacts.

Every raw shard and summary file is hash-checked with frozen identities (code, model, data, manifest, windows, query banks, and run metadata). Aggregate replay validates:

- protocol identity,
- raw call descriptors,
- raw trajectory equality,
- allocator continuity,
- storage formulas.

Key evidence identifiers: summary cbb5d861...c0401, scientific ledger 14fd1c0c...faf363.

7 Discussion and limitations

Under this controlled protocol, shared-document reuse improves parse-book scaling and preserves same-kernel exactness for resident counts up to $N = 32$.

Our claims are intentionally scoped:

- no global NVML or end-to-end system-memory conclusion,
- no claim of throughput or latency speedup,
- no inference about multi-document mixing, ragged batching, scheduler concurrency, Q8/Q4, or downstream quality metrics.

Next steps include explicit integration with production scheduling traces and broader quality traces under the same replay gates.

8 Conclusion

This work provides a strict, reproducible accounting of resident multi-request shared-document reuse under fixed same-kernel conditions. In this setting, parse-book demand declines linearly with N , with exactness preserved across tokens, logits, logical K/V, and GDN state.

At $N = 32$, parse-book controlled savings reach 2,720 MiB, and allocator peak allocated is reduced by approximately 2.661 GiB at the median over 8 ranks. We therefore report these values as scoped evidence, not as system-level deployment guarantees.

References

- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, Christopher Ré, Michael L. Soffa, C. W. Chen, Noam Shazeer, Quoc V. Le, et al. Flashattention: Fast and memory-efficient exact attention with io-awareness. arXiv preprint arXiv:2205.14135, 2022.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. Advances in Neural Information Processing Systems, 2017.
- vLLM Team. vllm: High-throughput and memory-efficient inference and serving for llms. <https://github.com/vllm-project/vllm>, 2024. vLLM unified attention and paged-key-value attention implementation.